International Journal of Research Publication and Reviews

Journal homepage: www.ijrpr.com ISSN 2582-7421

GenAI-Based SQL Chatbot for Natural Language Data Querying

Harshvardhan Gholap

UG Student, Parul Institute of Technology, Parul University, Vadodara

ABSTRACT:

Accessing relational data typically requires Structured Query Language (SQL), which creates a barrier for non-technical users and slows exploration analysis. This work presents a GenAI- powered SQL chatbot that enables natural-language interaction with both uploaded datasets (CSV/XLSX) and live Microsoft SQL Server databases. The system is designed as a unified platform that supports dynamic querying without predefined schemas or manual setup. Its architecture combines a FastAPI backend with a locally deployed LLaMA model through Ollama to translate user prompts into SQL.

To improve reliability and safety, the queries generated pass through a validation and correction pipeline that enforces read-only behavior and prevents unsafe operations. The platform includes schema-aware query generation, automatic multi-table join inference using foreign-key and column-level signals, and conversational context handling for follow-up questions. A result- grounded visualization module recommends charts based on actual query outputs to reduce hallucination risk.

In addition, the system is designed to provide a seamless and interactive user experience by allowing users to iteratively refine their queries through natural conversation. This reduces the dependency on technical expertise and enables faster data exploration and decision-making. The integration of local model deployment further ensures data privacy and minimizes reliance on external cloud services, making the solution suitable for organizations handling sensitive information.

User security is supported through JWT-based authentication and email OTP verification. The system demonstrates the feasibility of accurate, adaptive, and user-friendly database interaction, improving data accessibility for non-technical users while remaining useful for technical users in business intelligence, education, and decision-support workflows.

Introduction:

Organizations increasingly depend on data-driven decision making, yet practical data access is still limited by a major usability gap: most business users cannot write SQL. In many teams, non-technical stakeholders must wait for analysts or IT teams to fetch reports, transform data, and produce visual summaries. This dependency slows daily decision cycles, increases communication overhead, and often prevents timely exploration of important trends. The challenge becomes greater when data is distributed across structured databases and flat files such as CSV or Excel, where users need a unified way to ask questions without learning database syntax or schema details.

This work proposes a locally deployable GenAI-based SQL assistant for secure, efficient, and accessible data exploration. The system allows users to query data using natural language and supports both uploaded datasets (CSV/XLSX) and Microsoft SQL Server tables. Instead of requiring manually crafted query templates, the assistant dynamically interprets user intent, maps questions to available schema elements, and generates SQL queries that are validated before execution. The platform is built to be practical for real environments: users can view tabular outputs, receive chart-oriented insights, and export results in formats commonly needed for reporting workflows.

A key design principle of this project is privacy-preserving deployment. The language model is executed locally (LLaMA via Ollama), so data does not need to be sent to external cloud AI services for query generation. This architecture is especially relevant for organizations handling sensitive business data, where confidentiality and compliance requirements may restrict external data transfer. By combining local inference with controlled query execution, the system balances AI capability with operational trust.

The platform also emphasizes usability for non-technical users through conversational interaction and follow-up question handling. Users can ask iterative questions in natural language, refine filters, request summaries, and obtain visual representations without switching tools or writing code. At the same time, backend safety controls enforce read-only behavior and block unsafe operations, reducing the risk of accidental or malicious data modification. In this way, the solution
targets two parallel goals: broadening data access for non-technical users and preserving governance for technical teams.

Overall, this research addresses a practical and high-impact problem at the intersection of natural language processing, database systems, and human-computer interaction. The proposed system demonstrates how a local GenAI pipeline can reduce reporting latency, improve self-service analytics, and lower the skill barrier for data understanding while maintaining security and controllability in production-like settings.

The primary contributions of this work are as follows:

1. Design and implementation of a locally deployable natural-language-to-SQL assistant using LLaMA via Ollama, enabling privacy-preserving inference without external cloud dependency
2. Development of a unified data interaction framework that supports both Microsoft SQL Server tables and uploaded CSV/XLSX datasets within a single conversational interface.
3. Introduction of a schema-aware SQL generation mechanism with automatic multi-table relationship and JOIN inference, reducing manual schema mapping effort.
4. Integration of a multi-stage SQL reliability and safety pipeline for query cleaning, correction, schema validation, and strict read-only enforcement to block unsafe operations
5. Incorporation of conversational context handling follow-up queries, improving usability and continuity for non-technical users during iterative analysis.
6. Delivery of result-grounded analytics outputs, including tabular responses, chart recommendations, and downloadable CSV/Excel exports for practical reporting workflows.

Methodology:

The methodology defines the overall approach used to design and implement the proposed GenAI-powered Natural Language to SQL Chatbot system. The primary objective of this system is to enable users to interact with structured data using natural language, thereby eliminating the need for writing complex SQL queries.

The system follows pipeline-based architecture in which user input is processed through multiple stages including natural language understanding, SQL query generation, validation, execution, and result visualization. This approach ensures accuracy, security, and usability while handling real- world data scenarios.

The methodology is designed to support both technical and non-technical users by providing a conversational interface for data querying. It integrates modern technologies such as FastAPI, SQL Server, and Large Language Models (LLaMA via Ollama) to achieve efficient and secure data interaction.

1.1 Proposed Methodology.

The proposed system follows a structured workflow that transforms natural language input into meaningful analytical output. The process begins with the user entering a query in natural language through the user interface. This input is then processed by the backend system, where it is enriched with relevant schema information such as table names, column details, and relationships.

The overall working methodology of the proposed system has been illustrated in Fig. 1 This figure represents the complete pipeline, including user input processing, SQL query generation, validation, execution, and result visualization. The same methodology is explained in detail in this section to provide a deeper understanding of each stage.

In the next step, the processed input is passed to the Large Language Model (LLaMA), which is deployed locally using Ollama. The model interprets the user's intent and generates corresponding SQL query. This approach allows the system to dynamically handle different types of queries without requiring predefined templates or manual configuration.

Once the SQL query is generated, it is passed through a validation and correction pipeline. This stage ensures that the query is syntactically correct, adheres to SQL Server standards, and follows safety constraints such as read-only execution. Any errors or inconsistencies in the generated query are automatically corrected before execution.

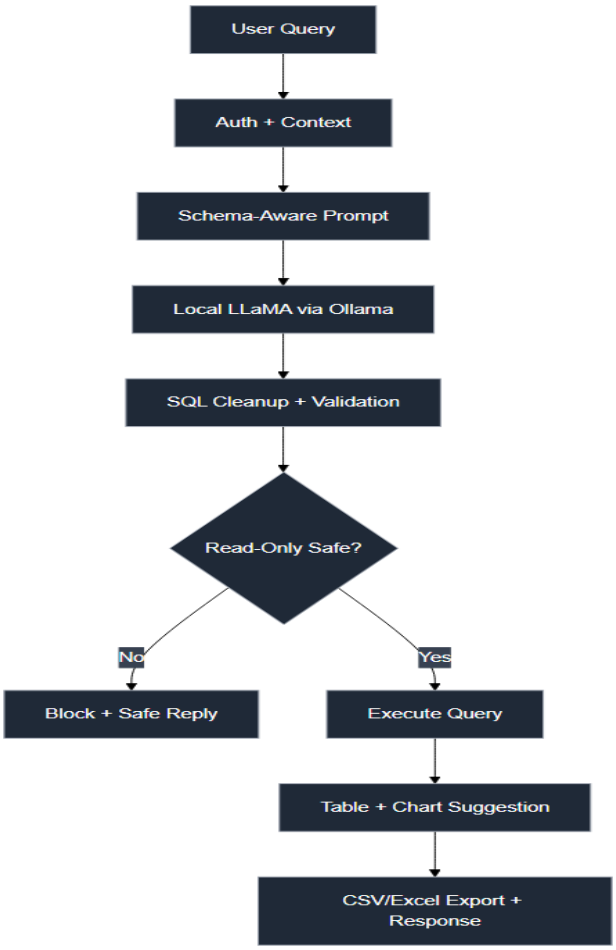

Fig. 1. Proposed methodology flow of the local GenAI SQL assistant.

After successful validation, the query is executed on the connected database, which may include SQL Server tables or uploaded datasets in CSV or Excel format. The system retrieves the results and prepares them for presentation to the user.

Finally, the output is displayed in both tabular and graphical formats. The system also supports chart generation based on the query results, enabling users to easily interpret the data. This complete workflow ensures efficient, secure, and user-friendly interaction between the user and the database system

1.2. System Architecture.

The system is designed using modular architecture consisting of multiple layers, each responsible for a specific functionality. Architecture ensures scalability, maintainability, and efficient communication between components.

The main layers include:

- Client Layer: Provides the user interface developed using React, allowing users to input queries and view results.
- API Layer: Built using FastAPI, it handles request routing, validation, and communication between frontend and backend.
- Intelligence Layer: Contains the LLaMA model running through Ollama, responsible for converting natural language into SQL queries.
- Data Layer: Includes SQL Server and uploaded datasets, which store and manage structured data.

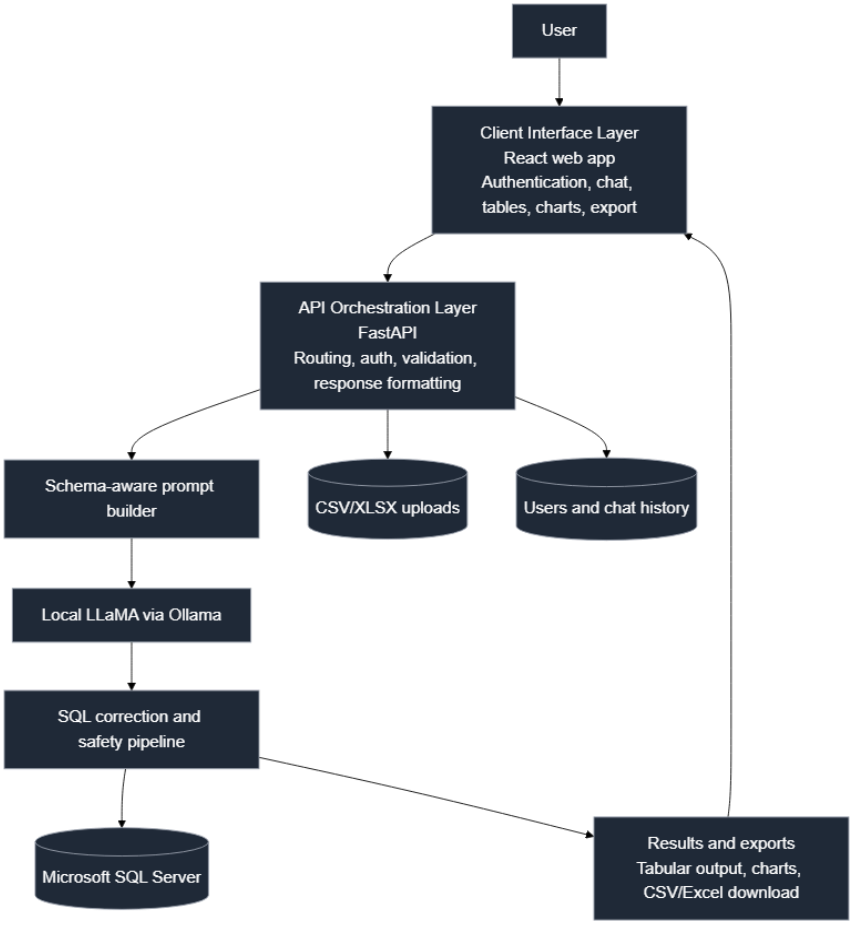

Fig. 2. High-level system architecture of the proposed system

1.3 Multi -Table JOIN Handling

The proposed system supports advanced query generation involving multiple tables, which is essential for performing real-world analytical operations. In many practical scenarios, data is distributed across multiple related tables, and meaningful insights can only be obtained by combining this data using JOIN operations.

However, writing JOIN queries manually requires a strong understanding of database schema and relationships, which is often difficult for non-technical users.

To address this challenge, the system implements an intelligent and automated multi-table JOIN handling mechanism. This mechanism uses a hybrid approach to identify relationships between tables and construct appropriate JOIN queries dynamically without requiring explicit user input.
The process begins by analysing the database schema, which includes information about table names, column names, data types, and relationships. The system first attempts to detect key foreign relationships between tables. These relationships provide the most reliable way to connect tables, as they are explicitly defined within the database.

Once the relationships are identified, the system dynamically constructs SQL JOIN queries based on the user's request. It determines the appropriate type of JOIN (such as INNER JOIN) and generates the required conditions to combine the tables correctly. The system also ensures that only relevant tables and columns are included in the query to maintain efficiency and accuracy.
To ensure reliability, the generated JOIN queries are passed through the validation pipeline, which checks for syntax correctness, schema alignment, and safe execution. This prevents errors and ensures that the query results are accurate and meaningful.

1.4 Output and Visualization

After the successful execution of the SQL query generated, the system produces output in a structured and user-friendly format. The primary form of output is a tabular representation, where the query results are displayed in rows and columns. This allows users to clearly view and understand the retrieved data without requiring any technical knowledge of SQL.
The system is designed to dynamically handle different types of query results, including simple data retrieval, filtered outputs, and aggregated results. The tabular format ensures that the data remains organized and easy to interpret, even when dealing with large datasets.

To maintain consistency and usability, the output is presented in a clean and structured interface, ensuring that both technical and non-technical users can easily interact with the system. The combination of tabular data, graphical visualization, and export functionality significantly improves the overall user experience. Output and visualization component plays a crucial role in transforming raw query results into meaningful insights, enabling users to make informed decisions effectively.

Overall, the output and visualization component enhances the interpretability of data and helps users make informed decisions.

Results

The results of the proposed "GenAI-based SQL Chatbot System" indicate that Generative AI can be effectively used to simplify database querying by converting natural language into structured SQL queries. The system was tested using multiple real-world queries, demonstrating both high accuracy and fast response time in generating results. The integration of AI with database systems significantly reduces the dependency on manual query writing and improves overall efficiency in data analysis.

2.1. System Performance and Accuracy.

The system was evaluated using different types of queries such as simple data retrieval, conditional queries, aggregation queries, and multi-table JOIN queries. The results clearly demonstrate that the automated AI-based approach reduces human effort and minimizes errors that are commonly observed in manual SQL query writing

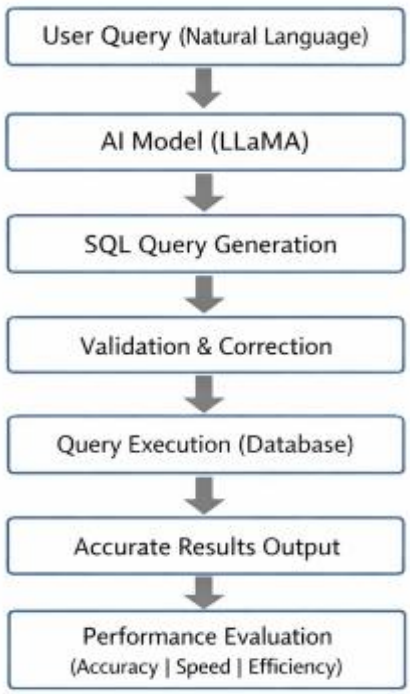

Fig. 3. System Performance and Accuracy Evaluation Flow

- Query Accuracy: The system generates highly accurate SQL queries using schema-aware prompt construction and validation pipeline.
- Efficiency: The results are generated in real-time due to optimized backend processing and local model deployment.
- Comparative Advantage: Compared to traditional methods, the system provides faster data access and eliminates the need for technical expertise.

2.2. Query Execution and Data Analysis

The system successfully executed SQL queries on both SQL Server databases and uploaded datasets such as CSV and Excel files. It was observed that the system can handle multiple query types efficiently and generate meaningful outputs

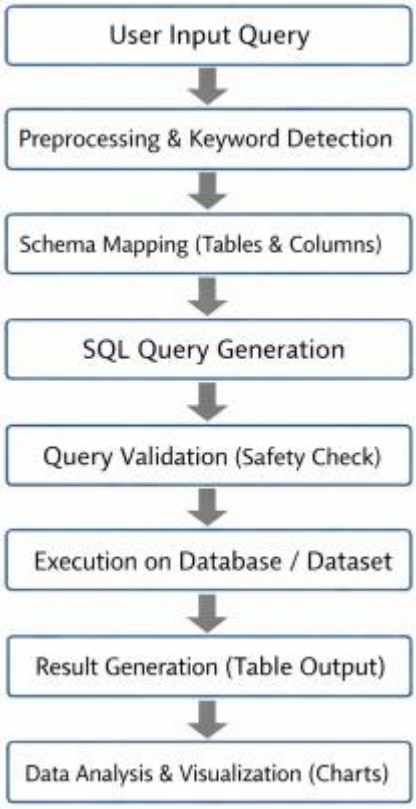

Fig. 4. Query Execution and Data Analysis Workflow

- Data Retrieval: The system accurately fetches data based on user queries.
- JOIN Handling: Automatic detection of relationships between tables improves multi-table query performance.
- Error Reduction: The validation pipeline ensures that incorrect or unsafe queries are corrected or blocked before execution.

2.3. Usability and Clinical Impact

The results highlight that the system is highly user-friendly and accessible, especially for non-technical users. The conversational interface allows users to interact with data in a natural and intuitive manner.

- **User Interface:** The system displays results in both tabular and graphical formats such as charts.
- **Accessibility:** Users without SQL knowledge can easily query databases using natural language.
- **Decision Support:** The system helps users quickly extract insights, improving decision-making processes

2.4. Visualization and Output Representation

The system provides graphical visualization of results using charts such as bar, line, and pie charts. This enhances data interpretation and allows users to understand trends and patterns effectively.

- **Graphical Representation:** Charts are generated dynamically based on query results.
- **Insight Generation:** Visualization helps in identifying patterns and making informed decisions.
- **Export Feature:** Results can be downloaded in CSV or Excel format for further use.

Conclusion

The GenAI-powered Natural Language to SQL Chatbot system successfully demonstrates an effective approach to simplifying data access for users. It enables natural language interaction with structured data, reducing the need for technical knowledge of SQL and making data analysis more accessible.

The system integrates Generative AI with database technologies to generate SQL queries and produce accurate results. Features such as schema-aware query generation, validation pipeline, and automatic JOIN handling improve the reliability and performance of the system. The support for conversational queries further enhances user interaction by allowing follow-up queries in a natural manner.

Additionally, the system provides visualization capabilities, user-friendly interface, and security mechanisms such as read-only enforcement, JWT authentication, and OTP verification. These features ensure both usability and data safety.

Although some limitations were observed, the system provides a strong foundation for future improvements. Overall, the project demonstrates a practical and scalable solution for intelligent data querying with real-world applicability.

REFERENCES:

List all the material used from various sources for making this project proposal
Research Papers:

1. Russell, S., & Norvig, P. (2010). Artificial Intelligence: A Modern Approach (3rd ed.). Prentice Hall.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.
4. Elmasri, R., & Navathe, S. (2016). Fundamentals of Database Systems (7th ed.). Pearson.
5. Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: Generating Structured Queries from Natural Language. arXiv preprint arXiv:1709.00103.
6. Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2020). RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. Proceedings of ACL.
7. Yu, T., Zhang, R., Yang, K., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing. EMNLP Conference.
8. Li, F., & Jagadish, H. V. (2014). NaLIR: An Interactive Natural Language Interface for Querying Relational Databases. ACM SIGMOD.
9. Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv preprint arXiv:2302.13971.